

SOLID et architecture hexagonale

Une application de parties de jeux de société

Anaël Chardan · IMT Atlantique · 2026

Une partie acceptée, une partie refusée

Brass: Birmingham accepte 2 à 4 joueurs.

Alice + Bob : partie valide.

Alice seule : refus métier.

À quel endroit doit vivre cette règle ?

Vous décidez, je code

Une prédiction avant chaque modification.

30 secondes pour comparer avec votre voisin.

Une vérification dans le code ou les tests.

Pas de configuration à faire pendant la séance.

Le vocabulaire utile

Type : la forme des données.

Interface : les opérations attendues.

Promise : un résultat qui arrive plus tard.

Test : un exemple dont on vérifie le résultat.

Le point de départ couplé

```
const { boardgameName, players } = JSON.parse(json);  
const game = { name: "Brass", min: 2, max: 4 };  
if (players.length < game.min) throw new Error();  
const row = {  
  game: game.name,  
  players_json: JSON.stringify(players)  
};
```

Changer JSON, la règle ou SQL touche le même endroit.

Les cinq principes SOLID

S Une responsabilité cohérente

O Une extension sur un axe prévu

L Un remplacement qui respecte le contrat

I Des interfaces adaptées aux consommateurs

D Des dépendances vers les besoins métier

S · Les raisons de changer

Le format HTTP change avec le client.

La règle change avec le besoin métier.

Le format de stockage change avec la base.

Quel changement justifie chaque séparation ?

S · La règle isolée

```
function validateCount(game: Game, players: string[]) {  
    if (players.length < game.min ||  
        players.length > game.max) {  
        throw new Error("Nombre invalide");  
    }  
}
```

Ce test a-t-il besoin d'un serveur ?

O · Le catalogue peut varier

Aujourd'hui : des jeux en mémoire.

Demain : BoardGameGeek.

La règle du nombre de joueurs reste la même.

Quel point de variation avons-nous choisi ?

O · Une extension par remplacement

```
const validate = makeValidator(fixtureCatalogue);  
  
// Même politique, autre fournisseur  
const validateLive = makeValidator(bggCatalogue);
```

Une nouvelle règle peut encore modifier le cas métier.

L · Un contrat observable

Jeu connu : un Boardgame.

Jeu inconnu : null.

Panne technique : CatalogUnavailable.

```
getBoardgameByName(name: string)  
: Promise<Boardgame | null>
```

L · Cela compile, mais cela casse

```
const brokenCatalogue: Catalogue = {  
  async find(name) {  
    throw new Error("Jeu inconnu");  
  }  
};
```

Que devient le code qui attend null ?

L · Un contrat commun

```
expect(await catalogue.find("Inconnu"))  
  .toBeNull();  
  
expect(await catalogue.find("Azul"))  
  .toMatchObject({ min: 2, max: 4 });
```

La signature seule ne suffit pas.

I · Une interface trop large

```
interface PlayRepository {  
    save(play: Play): Promise<void>;  
    all(): Promise<Play[]>;  
    deleteAll(): Promise<void>;  
    rebuildStatistics(): Promise<void>;  
}
```

De quoi le compteur de parties a-t-il besoin ?

I · Un besoin de lecture

```
interface PlayReader {  
  all(): Promise<Play[]>;  
}  
  
async function countPlays(reader: PlayReader) {  
  return (await reader.all()).length;  
}
```

D · Une dépendance vers la technique

```
// Dans le cas métier
import { PrismaClient } from "@prisma/client";

const db = new PrismaClient();
await db.play.create({ data: sqlRow });
```

Faut-il une base pour vérifier la règle ?

D · Le besoin appartient au métier

```
// domain/ports.ts
interface PlayWriter {
  save(play: Play): Promise<void>;
}

// Au démarrage, on fournit l'implémentation
const play = buildPlayAGame(catalogue, store);
```

DIP : qui définit le contrat ? DI : qui fournit l'objet ?

SOLID dans notre application

S Séparer les raisons de changer.

O Remplacer le catalogue sur un axe prévu.

L Respecter absence, résultat et erreurs.

I Demander seulement lecture ou écriture.

D Définir les ports du côté métier.

Les frontières de l'application

Métier : les parties valides et les cas d'usage.

Entrées : HTTP, CLI et tests.

Sorties : catalogue et stockage.

Une forme de dossier ne garantit pas la séparation.

Ports et adaptateurs

Appels à l'exécution



Les adaptateurs dialoguent via les contrats du domaine.

Les dépendances du code

```
infrastructure/http.ts
  imports domain/play-a-game.ts

infrastructure/prisma-store.ts
  imports domain/ports.ts

domain/*.ts
  imports seulement le domaine
```

Lecture : « ce fichier connaît le type de... »

Les appels pendant une requête

1. HTTP convertit le JSON en demande métier.
2. PlayAGame consulte le catalogue.
3. PlayAGame vérifie la règle.
4. PlayAGame enregistre via PlayWriter.
5. HTTP transforme le résultat en réponse.

Un appel sortant peut respecter une dépendance vers le métier.

Les objets métier

```
type Boardgame = {  
  name: string; bggId: string;  
  minNumberOfPlayers: number;  
  maxNumberOfPlayers: number;  
};  
type Play = {  
  boardgameName: string; bggId: string;  
  players: string[];  
};
```

Le cas d'usage hors ligne

```
const game = await inventory.getBoardgameByName(name);  
if (!game) throw new BoardgameNotFound();  
  
if (count < game.minNumberOfPlayers ||  
    count > game.maxNumberOfPlayers) {  
    throw new InvalidPlayerCount(min, max);  
}
```

La règle se vérifie sans HTTP et sans BGG.

Les limites font de bons exemples

1 joueur : refus

2 joueurs : accepté

4 joueurs : accepté

5 joueurs : refus

Jeu inconnu : erreur distincte

Que se passe-t-il si « $>$ » devient « $\geq$ » ?

Pause

10 minutes

À la reprise : la même règle derrière HTTP.

HTTP est un adaptateur

```
const parsed = requestSchema.safeParse(request.body);  
if (!parsed.success) return reply.code(400).send(...);  
  
const result = await play(parsed.data);  
return reply.code(200).send(result);
```

Le JSON reste à la frontière.

Des erreurs qui disent ce qui arrive

400 Le JSON ne correspond pas à la demande.

404 Le catalogue ne connaît pas le jeu.

422 Le nombre ou les noms sont invalides.

503 Le catalogue est indisponible.

500 Une erreur interne inattendue.

Prédiction : panne ou règle ?

Le catalogue répond 401.

- A. Le jeu est inconnu.
- B. Le catalogue est indisponible.
- C. La partie est valide.

30 secondes avec votre voisin.

Le catalogue externe

Mode de cours : catalogue en mémoire.

Mode live : BGG avec un jeton d'application.

Même port, mêmes règles métier.

Le réseau peut échouer sans changer la règle.

La traduction de BCG

```
<minplayers value="2" />
<maxplayers value="4" />

// Le domaine reçoit
{ minNumberOfPlayers: 2, maxNumberOfPlayers: 4 }
```

Le XML ne traverse pas le port.

Ce que chaque test vérifie

Métier : quelles parties sont valides ?

Contrat : les catalogues sont-ils substituables ?

Adaptateur : le XML devient-il le bon objet ?

Intégration : la base conserve-t-elle les données ?

Enregistrer par un port

```
interface PlayWriter {  
    save(play: Play): Promise<void>;  
}  
  
// Après toutes les validations  
await writer.save(play);  
return play;
```

À quel moment peut-on annoncer « créée » ?

Une partie refusée ne s'enregistre pas

```
await play(validRequest);  
await expect(play(invalidRequest)).rejects.toThrow();  
  
expect(await store.all()).toHaveLength(1);
```

Le retour de la fonction ne prouve pas une écriture.

La preuve par SQLite et Prisma

```
await client.play.create({ data: mappedPlay });  
  
// Puis une nouvelle connexion à la même base  
const rows = await store.all();
```

Même PlayWriter, stockage réellement persistant.

Une deuxième entrée : le terminal

```
npm run cli -- "Azul" Alice Bob

// L'adaptateur appelle le même cas d'usage
await play({ boardgameName, players });
```

Faut-il réécrire la règle du nombre de joueurs ?

Défi : deux fois la même personne

Nouvelle règle : chaque nom doit être unique.

« Alice » et « alice » désignent la même personne.

Les espaces autour du nom ne comptent pas.

Quels fichiers faut-il modifier ? Quels tests ajouter ?

La règle change une seule fois

```
const names = players.map(name => name.trim());  
const unique = new Set(  
    names.map(name => name.toLowerCase())  
);  
if (unique.size !== names.length) {  
    throw new InvalidParticipants();  
}
```

HTTP et CLI refusent la même partie.

Le coût de la séparation

Plus de fichiers et de contrats à comprendre.

Des conversions à maintenir et à tester.

Une frontière vaut son coût si elle protège un besoin.

Un petit script jetable aurait-il besoin de tout cela ?

Ce que vous savez maintenant décider

Où placer une règle métier.

Quel contrat rend un fournisseur remplaçable.

Quel test vérifie chaque frontière.

Comment ajouter une entrée sans dupliquer la règle.

Quelle dépendance supprimeriez-vous en premier ?